PRAKTEK TEKNOLOGI IOT II

LAPORAN PRAKTEK

TEKNOLOGI IOT II

Disusun dalam rangka memenuhi salah satu syarat kelengkapan nilai Praktikum

Mata Kuliah Teknologi IOT II

DISUSUN OLEH :

NAMA : Davina Anisa

NPM 25782007

PRODI : TEKNOLOGI REKAYASA INTERNET

KELAS : 3A

DOSEN : Dr. ir. Septafiansyah Dwi Putra, S. T.,M.T.

: Agus Ambarwari, S.Pd.,M.Kom.

4.5.1 Praktikum 1: *Real-Time Dashboard* & Sinkronisasi Tombol Fisik

Skenario: Anda memiliki lampu LED (simulasi aktuator/lampu cerdas) yang bisa dihidupkan melalui sakelar fisik (*push button*) di *breadboard*, ATAU melalui tombol di antarmuka web *smartphone* Anda. Keduanya harus tersinkronisasi dua arah secara *real-time*. Jika Anda menekan tombol fisik di meja praktikum, status visual indikator di antarmuka *smartphone* harus otomatis berubah seketika, begitu pula sebaliknya jika dikontrol dari web.

Langkah Kerja Rangkaian:

1. Hubungkan kaki **Anoda** (kaki panjang) LED ke pin **D6 (GPIO 12)**. Hubungkan kaki **Katoda** (kaki pendek) LED ke salah satu kaki **Resistor 220 Ohm**, lalu hubungkan kaki resistor lainnya ke pin **GND** NodeMCU *(sama seperti rangkaian LED pada Modul 3)*.
2. Hubungkan pin **DATA / OUT** DHT ke pin **D4 (GPIO 2)**. (VCC DHT ke 3V3, GND ke GND).
3. Pasang **Push Button** pada *breadboard*, melintasi parit tengah. Hubungkan salah satu kakinya ke jalur **3V3 (3.3V)**. Hubungkan kaki seberang tombol ke pin **D2 (GPIO 4)** NodeMCU. Pada titik kaki yang terhubung ke pin D2 tersebut, pasang salah satu kaki **Resistor 10k Ohm**, lalu hubungkan ujung kaki resistor lainnya ke jalur **GND** *(konfigurasi resistor pull-down eksternal seperti pada Modul 1)*.

Langkah Kerja Pemrograman:

Kode program ini cukup kompleks karena mengombinasikan *Front-End* (HTML, CSS, JavaScript) dengan *Back-End* (C++). Agar *real-time response* tombol tidak terganggu oleh sensor DHT yang butuh jeda baca lama, kita menerapkan teknik pemrograman *non-blocking* menggunakan fungsi millis().

1. Buat berkas baru (*New Sketch*). Tuliskan kode program berikut.

```
#include <ESP8266WiFi.h>
#include <ESPAsyncTCP.h>
#include <ESPAsyncWebServer.h>
#include <DHT.h>

const char* ssid = "NAMA_WIFI_ANDA";
const char* password = "PASSWORD_WIFI_ANDA";

// Konfigurasi Pin
const byte dhtPin = 2;        // D4 (GPIO 2)
const byte buttonPin = 4;     // D2 (GPIO 4)
const byte ledPin = 12;       // D6 (GPIO 12)

DHT dht(dhtPin, DHT11);

// Variabel Pelacak Status (State & Cache)
bool ledState = false;
String currentTemp = "--";
int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0;
const unsigned long debounceDelay = 50;
unsigned long lastTime = 0;

// Inisialisasi Async Web Server (port 80) & WebSocket (rute /ws)
AsyncWebServer server(80);
AsyncWebSocket ws("/ws");

// ---------------- HTML & JAVASCRIPT (FRONT-END) ----------------
```

```
const char index_html[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Real-Time IoT Web</title>
  <style>
    body { font-family: Arial; text-align: center; }
    .card { background: #f0f0f0; margin: 20px auto; padding: 20px; max-width: 300px; border-radius: 10px; }
    button { padding: 15px 30px; font-size: 20px; border-radius: 5px; cursor: pointer; color: white;}
    .btn-on { background-color: #4CAF50; }
    .btn-off { background-color: #f44336; }
  </style>
</head>
<body>
  <h1>Smart Room</h1>
  <div class="card">
    <h2>Suhu: <span id="tempValue">--</span> Celcius</h2>
  </div>
  <div class="card">
    <h2>LED: <span id="ledStatus">OFF</span></h2>
    <button id="toggleBtn" class="btn-off" onclick="toggleLed()">Turn ON</button>
  </div>

  <script>
    // Membuka terowongan koneksi WebSocket ke alamat IP ESP8266
    var gateway = `ws://${window.location.hostname}/ws`;
    var websocket;

    // Menjalankan inisiasi saat halaman pertama kali dimuat
    window.addEventListener('load', onLoad);
    function onLoad(event) { initWebSocket(); }

    function initWebSocket() {
      websocket = new WebSocket(gateway);
      websocket.onopen    = onOpen;
      websocket.onclose   = onClose;
      websocket.onmessage = onMessage;
    }

    function onOpen(event) { console.log('WebSocket Terkoneksi'); }
    function onClose(event) { setTimeout(initWebSocket, 2000); }

    // Dipanggil saat tombol di layar web ditekan
    function toggleLed(){
      websocket.send('toggle');
    }

    // Menangkap paket data JSON yang "didorong" (PUSH) oleh C++ NodeMCU
    function onMessage(event) {
      var dataObj = JSON.parse(event.data);

      // Menyuntikkan teks angka Suhu ke dalam id "tempValue" HTML
      if(dataObj.suhu !== undefined) {
```

```
      document.getElementById('tempValue').innerHTML = dataObj.suhu;
    }

    // Merombak UI Tombol & Teks sesuai status hardware
    if(dataObj.led !== undefined) {
      var btn = document.getElementById('toggleBtn');
      var status = document.getElementById('ledStatus');
      if(dataObj.led == "1"){
        status.innerHTML = "ON";
        btn.innerHTML = "Turn OFF";
        btn.className = "btn-on";
      } else {
        status.innerHTML = "OFF";
        btn.innerHTML = "Turn ON";
        btn.className = "btn-off";
      }
    }
  }
 </script>
</body>
</html>
)rawliteral";

// ---------------- BACK-END & WEBSOCKET LOGIC ----------------

// Fungsi menyebarkan paket data JSON ke SELURUH browser pengunjung
void notifyClients() {
 String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", ";
 jsonString += "\"suhu\":\"" + currentTemp + "\"}";
 ws.textAll(jsonString);
}

// Handler pesan masuk (dari klik browser)
void handleWebSocketMessage(void *arg, uint8_t *data, size_t len) {
 AwsFrameInfo *info = (AwsFrameInfo*)arg;
 if (info->final && info->index == 0 && info->len == len && info->opcode == WS_TEXT) {
  data[len] = 0;
  if (strcmp((char*)data, "toggle") == 0) {
   ledState = !ledState;
   notifyClients();
  }
 }
}

// Event handler WebSocket bawaan AsyncWebServer
void onEvent(AsyncWebSocket *server, AsyncWebSocketClient *client, AwsEventType type,
             void *arg, uint8_t *data, size_t len) {
 switch (type) {
  case WS_EVT_CONNECT:
   Serial.printf("Client WebSocket #%u terhubung\n", client->id());
   notifyClients(); // Kirim status terkini kepada klien yang baru buka web
   break;
  case WS_EVT_DISCONNECT:
   Serial.printf("Client WebSocket #%u terputus\n", client->id());
```

```
      break;
    case WS_EVT_DATA:
      handleWebSocketMessage(arg, data, len);
      break;
  }
}

void setup() {
  Serial.begin(115200);

  // Mengatur pin tombol sebagai input (dengan resistor pull-down eksternal)
  pinMode(buttonPin, INPUT);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
  Serial.println("\nIP Address: " + WiFi.localIP().toString());

  server.on("/", HTTP_GET, [](AsyncWebServerRequest *request){
    request->send_P(200, "text/html", index_html);
  });

  // Ikat handler WebSockets ke server utama
  ws.onEvent(onEvent);
  server.addHandler(&ws);
  server.begin();
}

void loop() {
  ws.cleanupClients();

  // 1. Eksekusi perangkat keras LED
  digitalWrite(ledPin, ledState ? HIGH : LOW);

  // 2. Baca Tombol Fisik (Debounce) tanpa nge-freeze program
  int reading = digitalRead(buttonPin);
  if (reading != lastButtonState) {
    lastDebounceTime = millis();
  }
  if ((millis() - lastDebounceTime) > debounceDelay) {
    if (reading != buttonState) {
      buttonState = reading;

      // Karena menggunakan resistor PULL-DOWN eksternal, tombol ditekan = HIGH
      if (buttonState == HIGH) {
        ledState = !ledState;
        notifyClients(); // PUSH DATA KE BROWSER SEKETIKA
      }
    }
  }
```

```
  lastButtonState = reading;

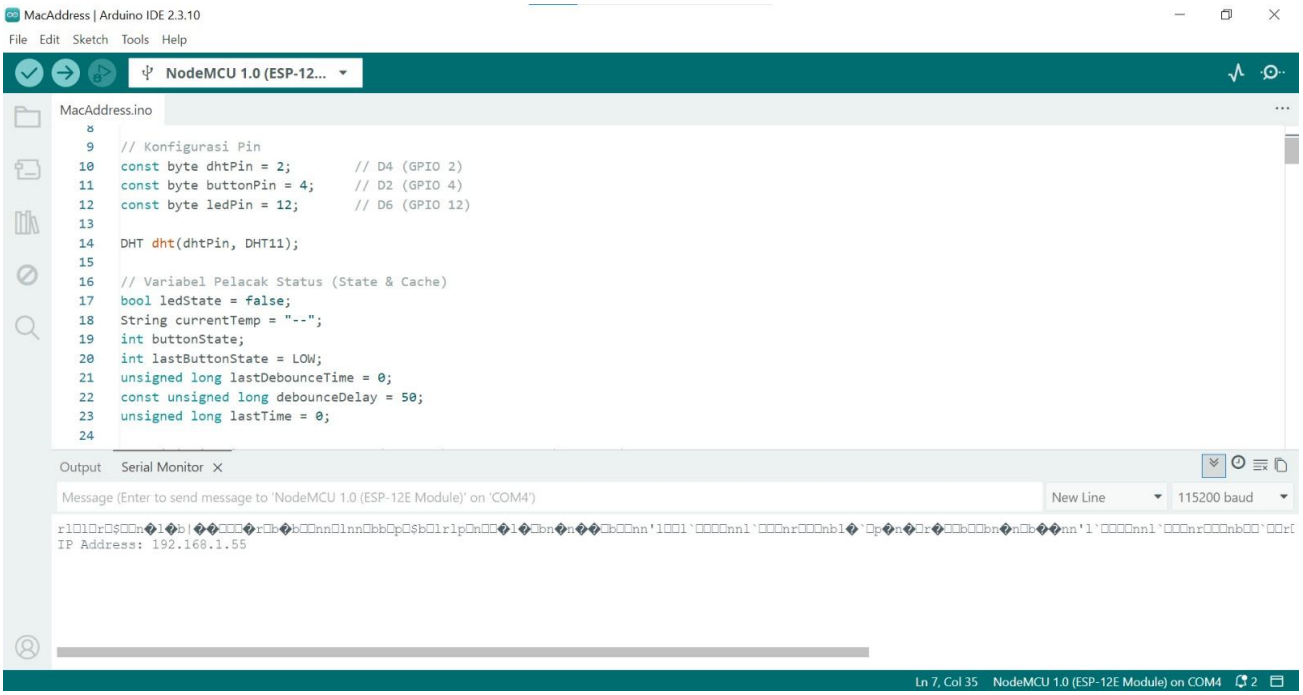
  // 3. Baca DHT setiap 3 detik di memori cache, tanpa memblokir CPU
  if ((millis() - lastTime) > 3000) {
    float t = dht.readTemperature();
    if(!isnan(t)) {
      currentTemp = String(t);
      notifyClients(); // Push pembaruan suhu otomatis
    }
    lastTime = millis();
  }
}
```

2. **Upload** program. Pastikan IP Address didapatkan dari Serial Monitor.
3. Buka IP Address tersebut di *browser* laptop Anda, dan **sekaligus** buka IP yang sama di *browser* ponsel (*smartphone*) Anda secara berdampingan.
4. **Uji 1 (Kendali Web Tersinkronisasi):** Klik tombol pada *browser* laptop Anda. Perhatikan bagaimana *browser* di ponsel Anda ikut berubah (*Turn ON*) secara presisi, dan lampu LED fisik ikut menyala!
5. **Uji 2 (Kendali Perangkat Keras):** Sentuh *push button* mekanik di meja Anda. Amati bahwa antarmuka *browser* di laptop maupun ponsel Anda secara serentak berganti status (*Turn OFF* menjadi *Turn ON*) **tanpa** terjadi proses *loading* atau *refresh* HTML halaman sama sekali! Kecepatan respon seketika ini adalah manfaat dari pemrograman *Asynchronous* dan pemisahan (*decoupling*) pembacaan memori sensor DHT.

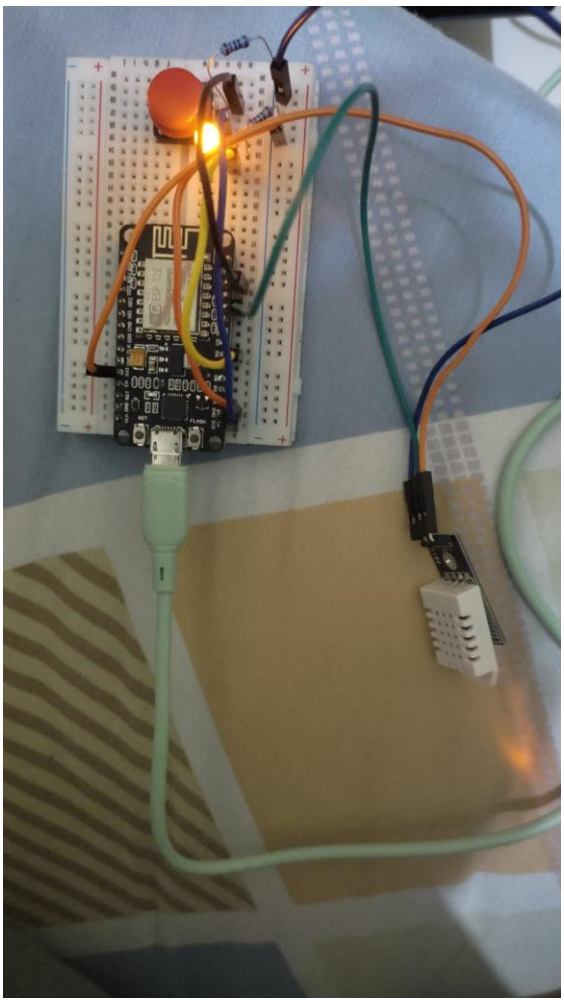
Penjelasan Singkat Kode Khusus:

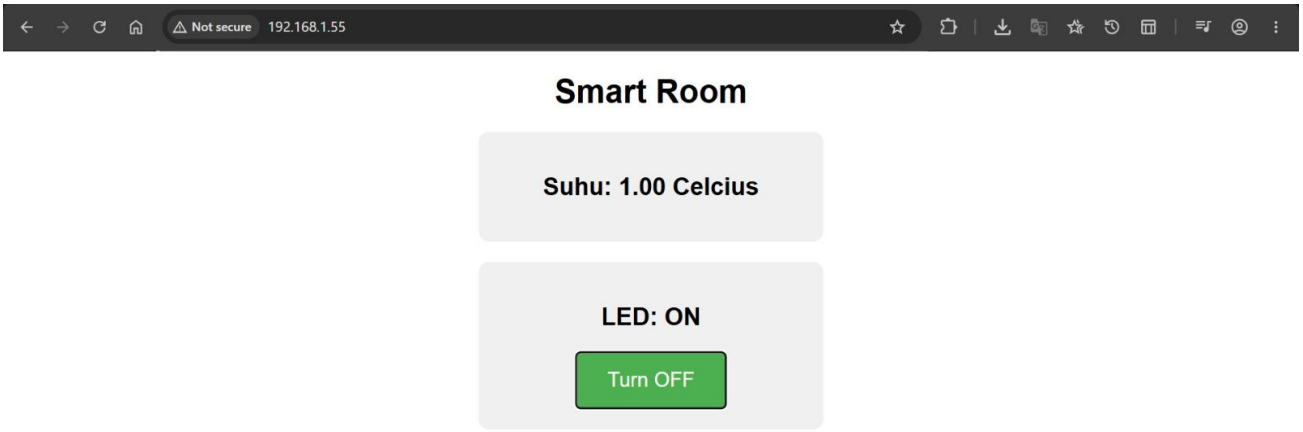
- ws.cleanupClients(): (Berada di siklus loop()). Fungsi penting dari pustaka *Asynchronous* ini bertugas membersihkan dan menutup sesi (*resource memori*) dari klien/*browser* yang sudah terputus atau menutup tab layarnya. Tanpa baris ini, memori RAM ESP8266 akan penuh (*memory leak*) dan berujung pada *Crash/Restart* jika terlalu banyak perangkat keluar-masuk membuka halaman *Dashboard*.
- pinMode(buttonPin, INPUT) dan if (buttonState == HIGH): Mempertahankan konfigurasi resistor *Pull-Down* eksternal 10k Ohm ke jalur GND (seperti pada Modul 1). Saat tombol dilepas, pin D2 membaca logika **LOW** (0V). Saat tombol ditekan, arus 3.3V mengalir sehingga terbaca logika **HIGH** yang memicu pembalikan status (*toggle*) LED secara instan.
- if ((millis() - lastTime) > 3000): Pemrograman *non-blocking*. Alih-alih membuat CPU tidur selama 3 detik menggunakan delay(3000) (yang akan membekukan respons WebSocket), kita menggunakan selisih penghitung jam internal CPU (millis()). Dengan demikian, CPU dapat terus siaga mengawasi sentuhan tombol fisik dan interaksi jaringan internet dengan kecepatan tinggi.
- var dataObj = JSON.parse(event.data): Sintaks JavaScript yang membedah teks kaku kiriman C++ berformat {"suhu": "30", "led": "1"} menjadi struktur data objek dinamis (*Dictionary*) untuk memperbarui elemen desain HTML dengan instan (*DOM Manipulation*).
- ws.textAll(): Perintah penyiaran (*broadcasting*) ampuh yang menembakkan/menyebarkan data secara seketika ke *seluruh* perangkat *browser* pengunjung (laptop, HP) secara bersamaan.

KODE PEMOGRAMAN:



RANGKAIAN DAN OUTPUTNYA:





TUGAS MANDIRI :

4.7 Tugas: Sistem Kendali Intensitas Cahaya (PWM Slider) via *WebSockets*

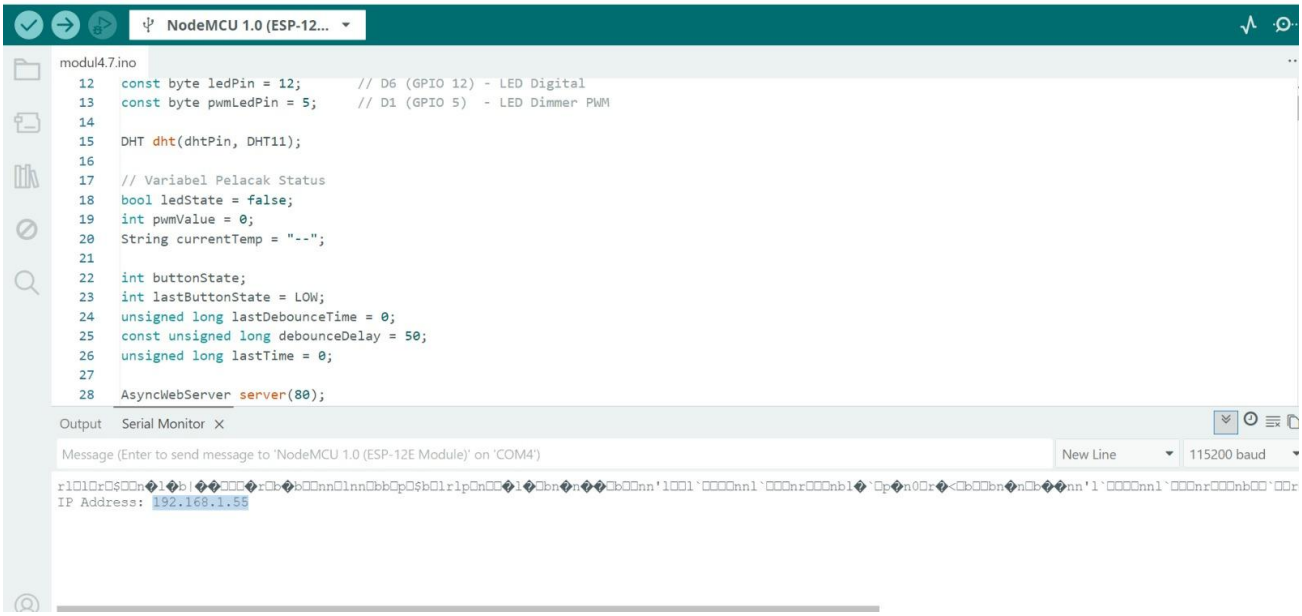
Sinyal Digital (*ON/OFF*) sangat berguna, namun bagaimana jika Anda ditugaskan merancang antarmuka penyesuai intensitas redup-terang lampu (*Dimmer*) dari *smartphone*?
Skenario Proyek (*Smart Lighting*): Anda akan mengeksplorasi penggunaan modulasi sinyal listrik analogWrite() (atau dikenal dengan fitur PWM) dan memadukannya dengan teknologi masukan elemen *Slider / Range* dari HTML5.

- **Tujuan:** Menguasai transmisi paket *String* WebSocket berkelanjutan (*continuous packet*) dari manipulasi antarmuka layar *slider* untuk merombak parameter persentase daya komponen aktuator.
- **Tingkat Kesulitan:** Lanjut
- **Instruksi Perangkaian:**
  i. Tambahkan sebuah komponen **LED** yang dilengkapi **Resistor 220 Ohm** (sebagai pengaman arus).
  ii. Hubungkan kaki Anoda (positif) sistem LED tersebut ke pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Front-End & Back-End):**
  i. Modifikasi desain kerangka web HTML Anda. Sisipkan elemen input *Slider* HTML. *(Hint sintaksis: <input type="range" min="0" max="1023" id="pwmSlider" onchange="sendPWM(this.value)">)*
  ii. Buat sebuah fungsi JavaScript bernama sendPWM(value) yang akan mengirimkan paket *string* teks berformat khusus (misalnya "pwm,512") ke NodeMCU melalui *websocket*.
  iii. Pada ruang fungsi C++ handleWebSocketMessage() di sisi server (NodeMCU), pecah (*parsing*) atau periksa string tersebut menggunakan fitur deteksi *substring* atau bahasa pemrosesan C++ lainnya.
  iv. Jika Server NodeMCU sukses mendeteksi dan mengambil angka murninya, panggil fungsi aktuator analogWrite(pinLED, nilaiPWM) untuk secara elegan meredupkan atau menerangkan LED LED fisik secara seketika sesuai gesekan layar Anda!
- **Kriteria Keberhasilan:**
  - Di layar gawai Anda, hadir tuas gulir horizontal (*Slider*).
  - LED di dunia nyata dapat diredupkan dan diterangkan secara bertahap saat Anda menggeser tuas *slider*, bebas dari lag (*real-time response*).
  - Kendali LED dan pembacaan Sensor *DHT* bawaan Praktikum 1 dipastikan masih bekerja berdampingan dengan sempurna.

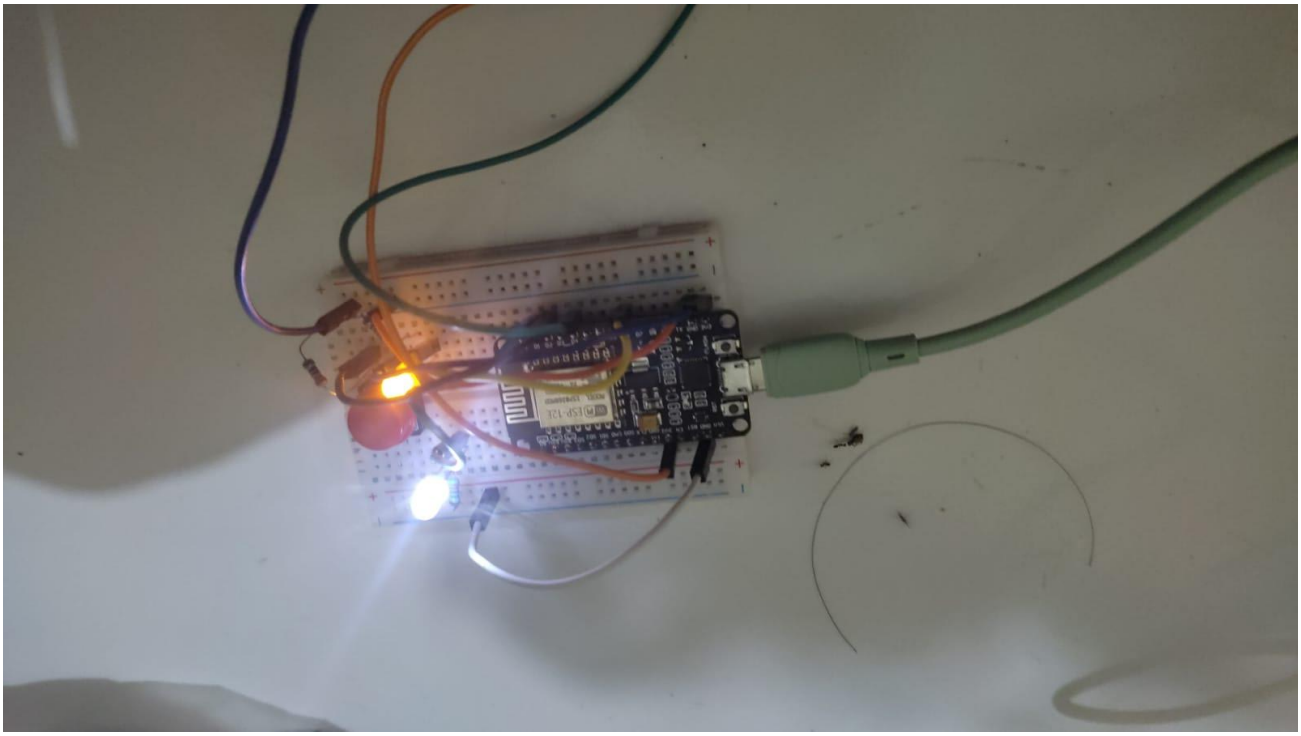
[PENTING!]
Instruksi Pengumpulan Tugas Praktikum Seluruh pengerjaan praktikum dan tugas (meliputi *source code* berformat .ino, dokumentasi foto/video sirkuit yang berhasil, tangkapan layar *Serial Monitor*, serta laporan praktikum tertulis .PDF) harus dikumpulkan dengan cara melakukan **commit dan push ke repositori**

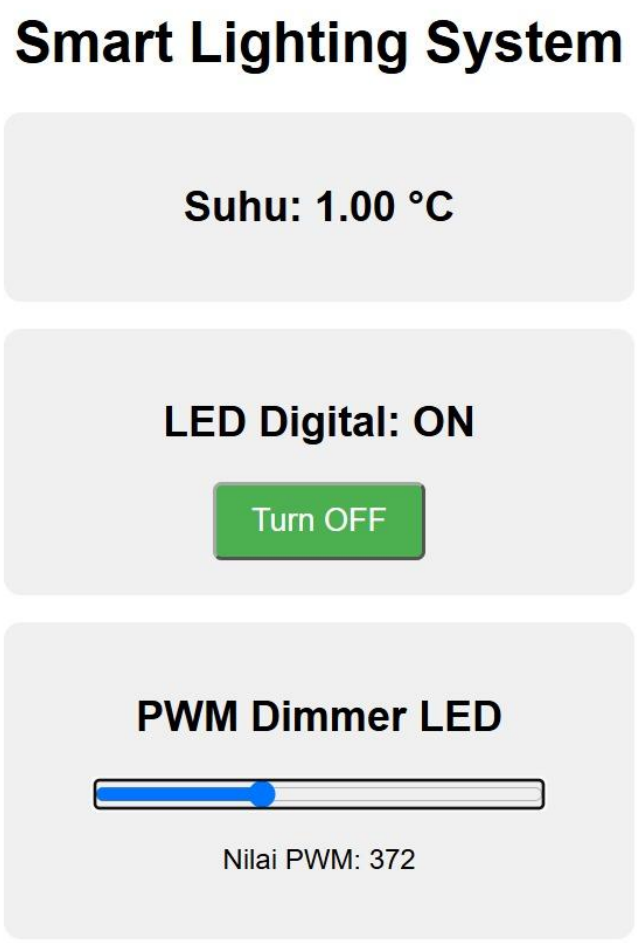
GitHub pribadi Anda masing-masing.
Tautkan/kumpulkan *URL repositori GitHub* Anda pada sistem manajemen pembelajaran (LMS) kampus sebagai bukti penyelesaian praktikum ini.

KODE PROGRAM :



RANGKAIAN DAN OUTPUTNYA:





LATIHAN:
1. Pembahasan dan Analisis Praktikum Sistem yang telah dibangun menggunakan arsitektur Asynchronous Web Server berbasis pustaka ESPAsyncWebServer dan protokol WebSockets.

• Keunggulan Dibanding HTTP Standar: Berbeda dengan modul sebelumnya yang menggunakan HTTP sinkron (request-response konvensional) di mana browser harus melakukan refresh atau menggunakan meta refresh (menyebabkan layar berkedip/flickering dan boros bandwidth), WebSockets membuka jalur pipa data persisten dua arah (full-duplex).

• Mekanisme Push Data: Ketika suhu berubah atau tombol fisik di breadboard ditekan, NodeMCU dapat secara proaktif mengirimkan (push) data berformat JSON ke browser tanpa menunggu permintaan dari klien. Hal ini membuat pembaruan antarmuka web menjadi instan dan mulus secara real-time.

• Pemrograman Non-Blocking: Penggunaan fungsi internal millis() menggantikan fungsi pemblokir seperti delay() memastikan mikrokontroler ESP8266 tetap siaga mengawasi interaksi jaringan dan tombol fisik tanpa mengalami freeze.

2. Jawaban Pertanyaan Latihan Latihan 1 (Teori Jaringan)
Soal: Mengapa teknologi koneksi WebSockets jauh lebih hemat bandwidth dan tidak membuat layar berkedip dibandingkan metode Meta Refresh HTTP HTML?

• Jawaban: WebSockets menggunakan satu koneksi TCP yang terbuka secara terus menerus (persistent connection). Setelah handshake awal berhasil, pertukaran data selanjutnya hanya mengirimkan paket data

kecil berukuran beberapa byte (seperti format JSON) melalui pipa yang sama. Sebaliknya, metode Meta Refresh HTTP mengharuskan browser mengunduh ulang seluruh struktur halaman HTML, CSS, dan aset gambar dari awal setiap kali ada pembaruan. Hal inilah yang menyebabkan layar berkedip (flickering) serta membebani jaringan karena transfer data yang berulang-ulang dan berukuran besar.

Latihan 2 (Analisis Waktu/Timer)

Soal: Apa bahaya sistematis yang akan terjadi pada komunikasi WebSockets jika Anda nekat memanggil sintaks delay(3000) di dalam siklus void loop()?

• Jawaban: Fungsi delay(3000) bersifat blocking (menghentikan total seluruh eksekusi CPU selama 3 detik). Akibatnya:

1. Mikrokontroler tidak akan bisa membaca status tombol fisik maupun memproses data sensor selama jeda tersebut.

2. Koneksi WebSocket akan mengalami timeout atau terputus karena server gagal merespons ping/pong atau heartbeat jaringan secara tepat waktu.

3. Sistem rentan mengalami Watchdog Timer (WDT) Reset yang membuat NodeMCU melakukan reboot otomatis akibat tugas utama jaringan terhenti terlalu lama.

Latihan 3 (Modifikasi Front-End) Soal: Analisislah fungsi JavaScript pada blok perusakan UI. Nama ID elemen HTML apa saja yang menjadi target injeksi/pengubahan teks?

• Jawaban: Berdasarkan skrip JavaScript pada praktikum, elemen ID yang menjadi target manipulasi DOM (Document Object Model) adalah:

1. tempValue: Target untuk menyuntikkan nilai suhu terbaru ke dalam elemen .

2. toggleBtn: Target untuk mengubah teks tombol (Turn ON / Turn OFF) serta kelas CSS tombol (btn-on / btn-off).

3. ledStatus: Target untuk mengubah teks status indikator LED (ON / OFF).

Latihan 4 (Pengembangan Telemetri Sensor - Kelembapan)

Soal: Modifikasi program agar sistem dapat membaca dan menampilkan data kelembapan secara real-time.

• Kode Modifikasi:

o Sisi C++ (Back-End): C++ // Tambahkan variabel penampung kelembapan di atas String

currentHum = "--"; // Di dalam timer non-blocking millis()

untuk DHT: if ((millis() - lastTime) > 3000) { float t = dht.readTemperature(); float h = dht.readHumidity(); // Baca kelembapan if(!isnan(t) && !isnan(h)) { currentTemp = String(t); currentHum = String(h); // Simpan ke cache notifyClients(); } lastTime = millis(); } // Modifikasi fungsi notifyClients(): void notifyClients() { String jsonString = "{\"led\":\"" + String(ledState ? 1 : 0) + "\", "; jsonString += "\"suhu\":\"" + currentTemp + "\", "; jsonString += "\"hum\":\"" + currentHum + "\"}"; // Tambah properti hum ws.textAll(jsonString); } o Sisi HTML (Front-End): HTML

Kelembapan: -- %

JavaScript
 o Sisi JavaScript (Front-End):
// Di dalam fungsi onMessage(event):
if(dataObj.hum !== undefined) { document.getElementById('humValue').innerHTML = dataObj.hum; }

4.8 Rangkuman

Pada Modul 4, Anda sukses meruntuhkan dinding pembatas arsitektur web tradisional yang statis. Melalui sintaks asinkronus dan keandalan protokol *WebSockets* ESPAsyncWebServer, kita membuat sebuah *Dashboard* modern di mana *Microcontroller* NodeMCU dapat mengirimkan data telemetri berformat JSON langsung ke web browser dan memperbarui halaman web di layar pengguna. Hal ini disempurnakan oleh perpaduan fundamental dari rutinitas penjadwalan millis() yang menghadirkan respons seketika antar multi-sensor dan interupsi pergerakan mekanik fisik. Kemampuan integrasi utuh (*Full-Stack*) inilah yang membedakan seorang pembuat prototipe IoT dengan seorang Perancang Sistem Industri.